

Глава 26. Артефакты

В этой главе:

- Артефакты, классы и их материализация
- Моделирование исполняемых программ и библиотек
- Моделирование таблиц, файлов и документов
- Моделирование исходного кода

Артефакты живут в материальном мире битов и потому являются важнейшими строительными блоками при моделировании физических аспектов системы. Артефакт – это физическая и заменяемая часть системы.

Артефакты используются для моделирования таких физических сущностей, которые могут располагаться на узле, – например, исполняемых программ, библиотек, таблиц, файлов и документов. Обычно артефакт представляет собой физическую группировку таких логических элементов, как классы, интерфейсы и кооперации.

Введение

Конечный результат работы строительной компании – здание в его материальном воплощении. Для визуализации, спецификации и документирования устройства жилого дома (расположения стен, дверей и окон, размещения систем электроснабжения и водопровода и т.д.), а также общего архитектурного стиля вы строите логические модели. Во время строительства стены, двери, окна и другие концептуальные сущности, данные вам в абстракциях, превращаются в реальные, физические объекты.

Все эти предварительные представления крайне необходимы. Если вы строите здание, стоимость разрушения и переделки которого практически равна нулю (например, собачью будку), то, вероятно, можете сразу приступить к физическому строительству, миновав этап логического моделирования. Если же речь идет о чем-то долговечном, то стоимость ошибок и переделок высока; в этом случае построение логической и физической моделей – разумный способ снизить степень риска.

Разница между строительством собачьей будки и небоскреба обсуждается в главе 1.

Аналогичным образом обстоит дело и с программными системами. Вы осуществляете логическое моделирование для визуализации, спецификации и документирования решений относительно словаря предметной области, а также структурных и поведенческих способов кооперации сущностей. Физическое же моделирование требуется для построения исполняемой системы. Если логические сущности «живут» в концептуальном мире, то физические пребывают в мире битов, то есть в конечном счете располагаются на физических узлах и могут быть исполнены непосредственно либо каким-то косвенным образом участвовать в работе исполняемой системы.

В UML все физические сущности моделируются как артефакты. Артефакт – это физическая сущность на уровне платформы реализации.

Многие операционные системы и языки программирования непосредственно поддерживают концепцию артефакта. Объектные библиотеки, исполняемые программы, компоненты .NET и Enterprise Java Beans, – все это примеры артефактов, которые могут быть представлены на UML. Но артефакты могут быть использованы и для представления других сущностей, которые принимают участие в работе исполняемых программ, – таких, как таблицы, файлы и документы.

В UML артефакты изображаются так, как показано на рис. 26.1. Эта каноническая нотация позволяет визуализировать артефакт независимо от операционной системы и языка программирования. Используя стереотипы – один из механизмов расширения UML, можно приспособить эту нотацию для представления специфических разновидностей артефактов.

Стереотипы обсуждаются в главе 6.



Рис. 26.1. Артефакты

Базовые понятия

Артефакт – это физическая часть системы, которая существует на уровне платформы реализации. Изображается в форме прямоугольника с ключевым словом «artifact» внутри.

Имена

Каждый артефакт должен обладать *именем*, отличающим его от других артефактов. Имя представляет собой текстовую строку; взятая в отдельности, она называется *простым именем*. Имя,

Имя артефакта должно быть уникальным в пределах включающего его узла.

предваренное именем пакета, в котором находится данный артефакт, называется *квалифицированным*. Обычно, изображая артефакт, указывают только его имя. Как и в случае с классами, вы можете дополнять пиктограммы артефактов помеченными значениями или дополнительными разделами, чтобы уточнить подробности (см. рис. 26.2).

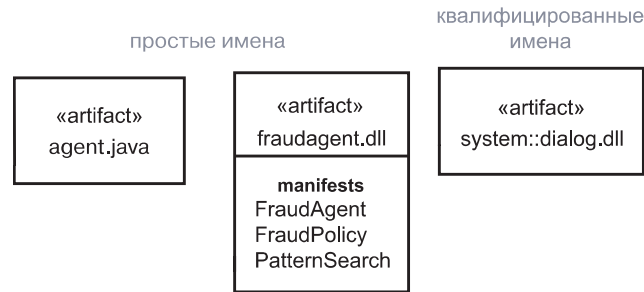


Рис. 26.2. Простые и квалифицированные имена артефактов

На заметку. Имя артефакта представляет собой текст, содержащий в любом количестве буквы латинского алфавита, цифры и ряд знаков препинания (за исключением таких, как двоеточия, которые применяются для отделения имени артефакта от имени включающего его пакета). Имя может записываться в несколько строк. На практике в качестве имен артефактов выступают существительные, взятые из словаря реализации и, в зависимости от целевой операционной системы, включающие расширения (такие как `java` или `dll`).

Артефакты и классы

И классы, и артефакты являются классификаторами. Однако между ними есть существенная разница:

- классы представляют логические абстракции, а артефакты – физические сущности, которые существуют в мире битов. В силу этого артефакты могут располагаться на узлах, а классы – нет;
- артефакты представляют физическую упаковку битов на платформе реализации;
- классы могут иметь атрибуты и операции. Артефакты могут реализовывать классы и методы, но сами по себе не имеют атрибутов и операций.

Первое отличие наиболее важно. Когда моделируется система, решение о том, нужно ли использовать классы или артефакты,

сводится к простому выбору: если моделируемая вами сущность «живет» непосредственно на узле, применяйте артефакт; в противном случае используйте класс. Подтверждением тому служит и второе отличие.

Третье отличие предполагает связь между классами и артефактами. В частности, артефакт – это физическая реализация набора логических элементов, таких как классы и кооперации. Как показано на рис. 26.3, связь между артефактом и классами, которые он реализует, может быть показана явно с помощью материализации.

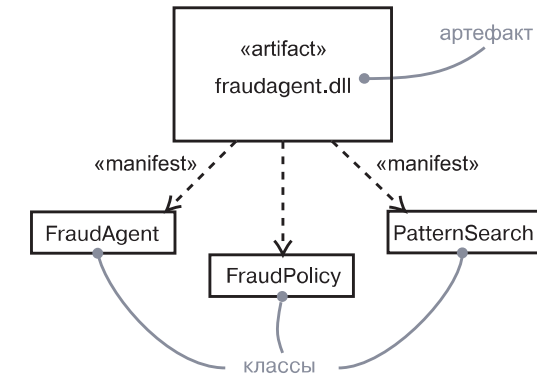


Рис. 26.3. Артефакты и классы

Виды артефактов

Можно выделить три вида артефактов:

1. *Артефакты размещения* (deployment artifacts) необходимы и достаточны для формирования исполняемой системы. Примеры – динамические библиотеки (DLL) и исполняемые программы (EXE). Определение артефакта UML достаточно широко, чтобы вместить классические объектные модели, такие как .NET, CORBA и Enterprise Java Beans, а равно и альтернативные объектные модели (возможно, включающие динамические Web-страницы, таблицы баз данных, исполняемые программы, использующие собственные коммуникационные механизмы);
2. *Артефакты рабочих продуктов* (work product artifacts), по существу, представляют собой результаты процесса разработки. Примеры – файлы исходного кода и файлы данных, из которых создаются артефакты размещения. Артефакты такого рода напрямую не участвуют в исполняемой системе, но являются продуктами разработки, используемыми для ее создания;

Классы обсуждаются в главах 4 и 9, взаимодействия – в главе 16.

3. *Артефакты исполнения* (execution artifacts) создаются в результате работы системы. Примеры – объект .NET, который создается из DLL.

Стандартные элементы

Механизмы расширения UML обсуждаются в главе 6.

Механизмы расширения UML применимы к артефактам. Чаще всего для расширения свойств артефактов (таких, например, как спецификация версии артефакта разработки) используются помеченные значения, а для определения новых видов артефактов (например, специфичных для конкретной операционной системы) – стереотипы.

Перечислим predefined стандартные стереотипы UML, применимые к артефактам:

1. Executable – специфицирует артефакт, который может быть выполнен на узле;
2. Library – специфицирует статическую или динамическую библиотеку объектов;
3. File – специфицирует артефакт, который представляет документ, содержащий исходный код или данные;
4. Document – специфицирует артефакт, представляющий документ.

Другие стереотипы могут быть определены для конкретных платформ и систем.

Типичные приемы моделирования

Моделирование исполняемых программ и библиотек

Чаще всего цель применения артефактов заключается в моделировании артефактов размещения, составляющих реализацию системы. Если вы устанавливаете простую систему, реализация которой состоит всего лишь из одного исполняемого файла, нет необходимости ни в каком моделировании артефактов. Если же система состоит из нескольких исполняемых программ и ассоциированных с ними объектных библиотек, моделирование артефактов помогает визуализировать, специфицировать, конструировать и документировать принятые решения относительно ее физической организации. Моделирование артефактов еще более важно, если требуется контролировать версии и управлять конфигурацией частей системы.

На подобные решения влияет и топология целевой системы (см. главу 27).

Для большинства систем артефакты размещения основываются на решениях относительно того, как должна быть сегментирована физическая реализация системы. На такие решения влияет множество соображений технического характера (в частности, выбранные средства операционной системы), удобства управления конфигурацией (например, вероятность изменения некоторых частей системы по прошествии времени), а также неоднократного использования (повторное применение ряда артефактов другими системами).

Чтобы смоделировать исполняемые программы или библиотеки, необходимо:

- ❑ Идентифицировать разбиение физической системы на части. Рассмотреть влияние технических требований, требований управления конфигурацией и повторного использования.
- ❑ Смоделировать любые исполняемые программы и библиотеки как артефакты, используя соответствующие стандартные элементы. Если ваша реализация представляет новые виды артефактов, создайте специальные стереотипы.
- ❑ Обращать внимание на соединения частей системы, моделируя интерфейсы, которые одни артефакты используют, а другие реализуют.
- ❑ Смоделировать связи между исполняемыми программами, библиотеками и интерфейсами для реализации вашего замысла. Чаще всего понадобится моделировать зависимости между этими частями, чтобы визуализировать воздействие изменений.

В качестве примера на рис. 26.4 показан набор артефактов, составляющих персональный инструмент измерения производительности, который запускается на отдельном компьютере. Здесь изображены одна исполняемая программа (animator.exe) и четыре библиотеки (dlog.dll, wframe.dll, render.dll и raytrace.dll); все они используют стандартные элементы UML, описывающие, соответственно, исполняемые программы библиотеки. Кроме прочего, данная диаграмма представляет зависимости между артефактами.

По мере роста ваших моделей вы обнаружите, что многие артефакты, концептуально и семантически близкие, имеют тенденцию собираться в группы. Для моделирования таких кластеров артефактов в UML можно использовать пакеты.

Для крупных систем, которые размещаются на нескольких компьютерах, может понадобиться моделирование способа распределения артефактов с указанием узлов, на которых они находятся.

Пакеты обсуждаются в главе 12. Моделирование размещения обсуждается в главе 27.

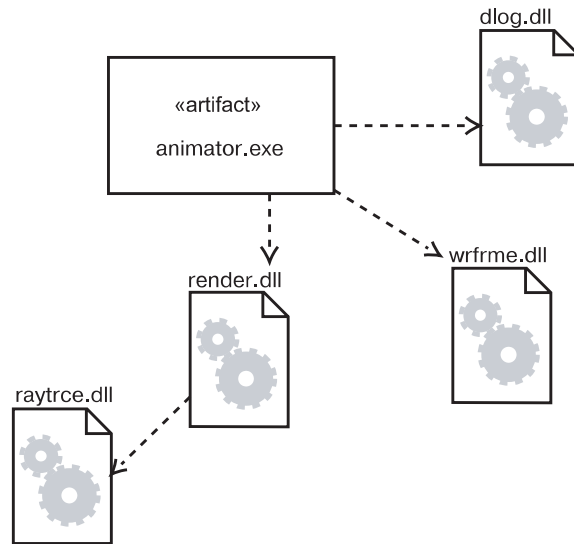


Рис. 26.4. Моделирование исполняемых программ и библиотек

Моделирование таблиц, файлов и документов

Моделирование исполняемых программ и библиотек, составляющих физическую реализацию системы, полезно, но часто обнаруживается, что существует множество вспомогательных артефактов, которые не являются ни исполняемыми файлами, ни библиотеками, хотя не менее важны для физического размещения системы. Например, реализация может включать файлы данных, вспомогательную документацию, скрипты, файлы протоколов, файлы инициализации, а также файлы, используемые при установке/удалении системы. Моделирование подобных артефактов – важная часть управления конфигурацией системы. К счастью, для этого хорошо подходят артефакты UML.

Чтобы смоделировать таблицы, файлы и документы, необходимо:

- ❑ Идентифицировать вспомогательные артефакты, являющиеся частью физической реализации системы.
- ❑ Смоделировать эти сущности в виде артефактов. Если ваша реализация представляет новые виды артефактов, понадобится ввести соответствующие стереотипы.
- ❑ При необходимости обозначить связи между этими вспомогательными артефактами и другими исполняемыми программами,

Логическое и физическое моделирование баз данных обсуждается в главах 8 и 30 соответственно.

библиотеками и интерфейсами системы. Чаще всего оказывается важным моделирование зависимостей между этими частями, позволяющее визуализировать влияние изменений.

Обратимся к рис. 26.5, который базируется на рис. 26.4. Здесь показаны таблицы, файлы и документы, являющиеся частями установленной системы, окружающими исполняемый файл `animator.exe`. Имеются в наличии один документ (`animator.hlp`), один простой файл (`animator.ini`) и одна таблица базы данных (`shapes.tbl`). Также данный пример иллюстрирует некоторые пользовательские стереотипы и пиктограммы артефактов.

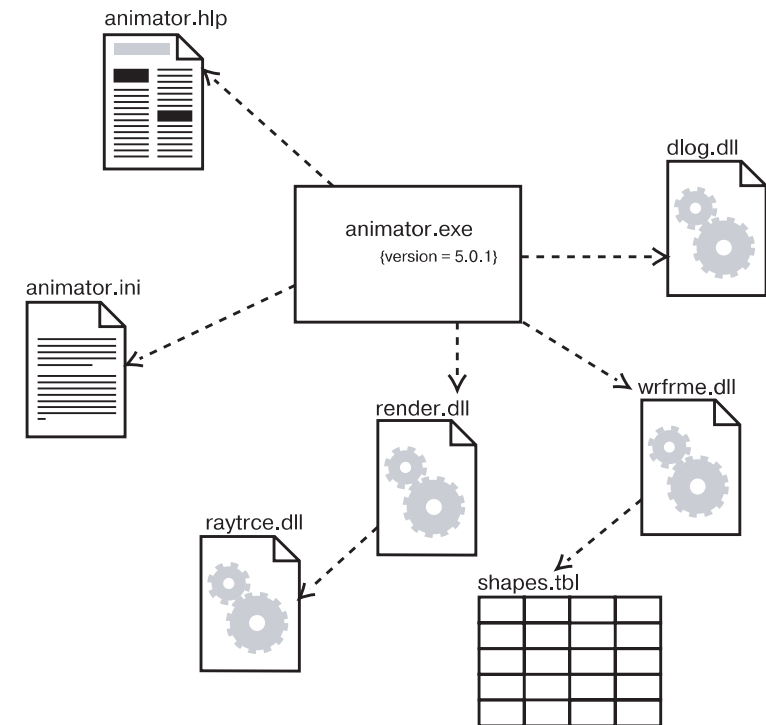


Рис. 26.5. Моделирование таблиц, файлов и

Моделирование баз данных значительно усложняется, когда вы начинаете иметь дело со множеством таблиц, триггеров и хранимых процедур. Чтобы визуализировать, специфицировать, конструировать и документировать эти средства, вам придется смоделировать логическую схему наряду с физической базой данных.

Моделирование исходного кода

Главное назначение артефактов – моделирование физических частей реализации. Второе по важности их применение – моделирование конфигурации всех файлов исходного кода, которые нужны средству разработки для построения исполняемых артефактов. Эти файлы представляют собой артефакты продуктов процесса разработки.

Моделирование исходного кода в графическом виде полезно, в частности, для визуализации зависимостей компиляции между файлами исходного кода, а также для управления расщеплением и слиянием групп этих файлов при распараллеливании и соединении самого процесса разработки. В этом смысле артефакты UML могут быть графическим интерфейсом инструментов управления конфигурацией и контроля версий.

Для большинства систем файлы исходного кода создаются с учетом решений, принятых вами относительно того, как должен быть сегментирован исходный код для нужд среды разработки. Эти файлы используются для хранения деталей реализации классов, интерфейсов, коопераций и других логических элементов – промежуточной фазы процесса создания физических бинарных артефактов, которые производятся из таких элементов инструментальными средствами разработки. Большей частью подобные инструменты определяют стиль организации (часто – один или два файла на класс), но все же для разработчика полезно визуализировать связи между этими файлами. Решения относительно того, как организовать исходные файлы в пакеты и как управлять их версиями, принимаются на основе выбранных способов управления изменениями.

Чтобы смоделировать исходный код, необходимо:

- ❑ С учетом ограничений, накладываемых средствами разработки, смоделировать файлы, используемые для хранения деталей реализации логических элементов, наряду с их зависимостями компиляции.
- ❑ Если важно, чтобы модели были согласованы с инструментами разработки и управления конфигурацией, для каждого файла, подверженного управлению конфигурацией, включить помеченные значения – такие как информация о версии, авторе и помещении файла в репозиторий / извлечении из репозитория.
- ❑ По возможности позволять инструментам разработки управлять связями между этими файлами и использовать UML только для того, чтобы визуализировать и документировать такие связи.

На рис. 26.6 показаны некоторые файлы исходного кода, используемые для построения библиотеки `render.dll` из предыдущих примеров. Рисунок включает четыре заголовочных файла (`render.h`, `engine.h`, `poly.h` и `colortab.h`), которые представляют исходный код спецификации определенных классов. Также имеется файл реализации (`render.cpp`), представляющий реализацию одного из этих заголовков.

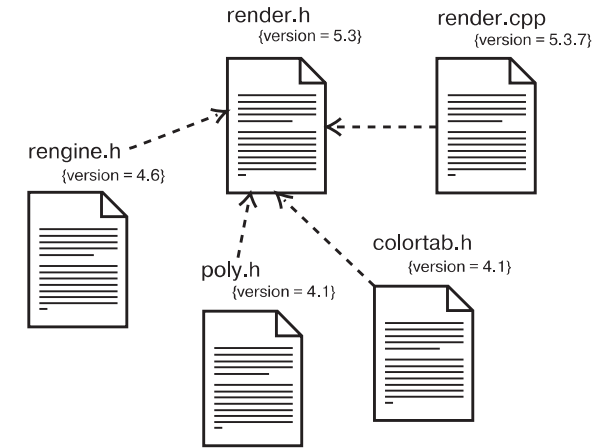


Рис. 26.6. Моделирование исходного кода

*Пакеты
обсуж-
даются
в главе 12.
Связь трас-
сировки
как разно-
видность
зависимости
рассмат-
ривается
в главах 5
и 10.*

По мере развития ваших моделей вы обнаружите, что многие файлы исходного кода, концептуально и семантически связанные, имеют тенденцию собираться в группы. Как правило, инструменты разработки будут размещать их в разных директориях. В UML для моделирования подобных кластеров исходного кода можно использовать пакеты.

Кроме того, в UML есть возможность визуализировать отношения между классом и его файлом исходного кода, а также, в свою очередь, связь файла исходного кода с исполняемой программой или библиотекой посредством трассировки (trace relationship). Однако необходимость в такой детализации при моделировании возникает редко.

Советы и подсказки

Работая с артефактами в UML, важно помнить о том, что вы осуществляете моделирование в физическом измерении. Хорошо структурированный артефакт обладает следующими характеристиками:

- ❑ непосредственно реализует набор классов, работающих совместно для обслуживания определенной семантики интерфейсов, — экономно и элегантно;
- ❑ слабо связан с другими артефактами.

Моделирование непрограммных сущностей обсуждается в главе 4, пять видов системной архитектуры описаны в главе 2.

Глава 27. Размещение

В этой главе:

- Узлы и соединения
- Моделирование процессоров и устройств
- Моделирование распределения артефактов
- Разработка систем

Узлы, равно как и артефакты, существуют в материальном мире и являются важными строительными блоками при моделировании физических аспектов системы. Узел — это физический элемент, который существует во время выполнения и представляет вычислительный ресурс, обычно обладающий как минимум некоторым объемом памяти, а зачастую также и процессором.

Узлы используются для моделирования топологии аппаратных средств, на которых работает система. Как правило, узел — это процессор или устройство, на котором могут быть размещены артефакты.

Хорошо спроектированные узлы точно соответствуют словарю аппаратного обеспечения области решения.

Введение

Артефакты, которые вы разрабатываете или повторно используете в программной системе, должны быть размещены на какой-то аппаратуре, иначе они не смогут выполняться. Собственно, программная система и состоит из этих двух частей: программного и аппаратного обеспечения.

При проектировании архитектуры программной системы приходится рассматривать как логические, так и физические аспекты. К логическим элементам относятся такие сущности, как классы, интерфейсы, кооперации, взаимодействия и автоматы, а к физическим — артефакты (представляющие физическую упаковку логических сущностей) и узлы (представляющие аппаратуру, на которой размещаются и выполняются артефакты).

*Стереотипы
обсуждаются
в главе 6.*

Графическое изображение узла в UML показано на рис. 27.1. Это каноническое обозначение позволяет визуализировать узел, не конкретизируя стоящей за ним аппаратуры. С помощью стереотипов – одного из механизмов расширения UML – можно адаптировать эту нотацию для представления конкретных процессоров и устройств.

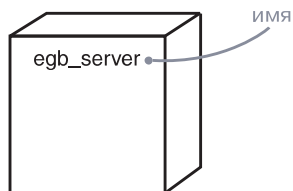


Рис. 27.1. Узлы

На заметку. Язык UML предназначен главным образом для моделирования программных систем, хотя в сочетании с языками моделирования аппаратных средств типа VHDL он может оказаться весьма полезным и при моделировании аппаратных систем. Кроме того, UML обладает достаточной выразительной мощностью для моделирования топологии автономных (stand-alone), встроенных (embedded), клиент-серверных (client-server) и распределенных (distributed) систем.

Базовые понятия

Узел (Node) – это физический элемент, который существует во время выполнения и представляет вычислительный ресурс, обычно обладающий как минимум некоторым объемом памяти, а зачастую и процессором. Изображается в виде куба.

Имена

*Имя узла
должно быть
уникальным
внутри вклю-
чающего его
пакета, это
обсуждается
в главе 12.*

У каждого узла должно быть имя, отличающее его от прочих узлов. Имя представляет собой текстовую строку. Взятое само по себе, оно называется *простым именем*. К *квалифицированному имени* спереди добавлено имя пакета, в котором находится данный узел.

Обычно при изображении узла указывают только его имя, как видно из рис. 27.2. Но, как и в случае с классами, вы можете снабжать узлы помеченными значениями или дополнительными разделами, чтобы акцентировать детали.

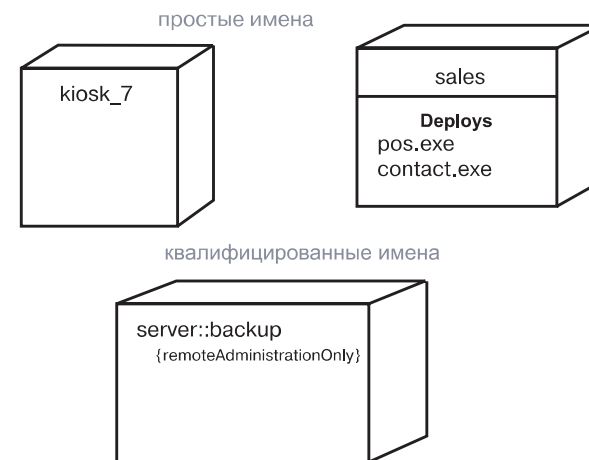


Рис. 27.2. Узлы с простыми и квалифицированными именами

На заметку. Имя узла может состоять из любых букв латинского алфавита, цифр и некоторых знаков препинания (за исключением таких, как двоеточие, которое применяется для отделения имени узла от имени включающего пакета). Имя может занимать несколько строк (количество символов не ограничено). На практике для именования узлов используют одно или несколько существительных, взятых из словаря реализации.

Узлы и артефакты

*Артефакты
обсуждаются
в главе 26.*

Во многих отношениях узлы подобны артефактам. Те и другие наделены именами, могут участвовать в связях зависимости, обобщения и ассоциации, бывают вложенными, могут иметь экземпляры и вступать во взаимодействия. Однако между ними есть и существенные различия:

- ❑ артефакты принимают участие в работе системы, а узлы – это сущности, на которых работают артефакты;
- ❑ артефакты представляют физическую упаковку логических элементов, узлы представляют физическое размещение артефактов.

Первое из этих отличий самое важное. Суть его проста: узлы исполняют артефакты, артефакты работают на узлах.

Зависимости
обсуждаются
в главе 5
и 10.

Второе различие предполагает наличие некоторой связи между классами, артефактами и узлами. В самом деле, артефакт — это материализация множества логических элементов, таких как классы и кооперации, а узел — место, на котором размещены артефакты. Класс может быть реализован одним или несколькими артефактами, а артефакт, в свою очередь, размещен в одном или нескольких узлах. Как показано на рис. 27.3, связь между узлом и артефактами, которые на нем размещены, может быть явно изображена с помощью вложенности. Как правило, вам не придется визуализировать такие связи. Лучше обозначать их как часть спецификации узла, используя, например, таблицу.

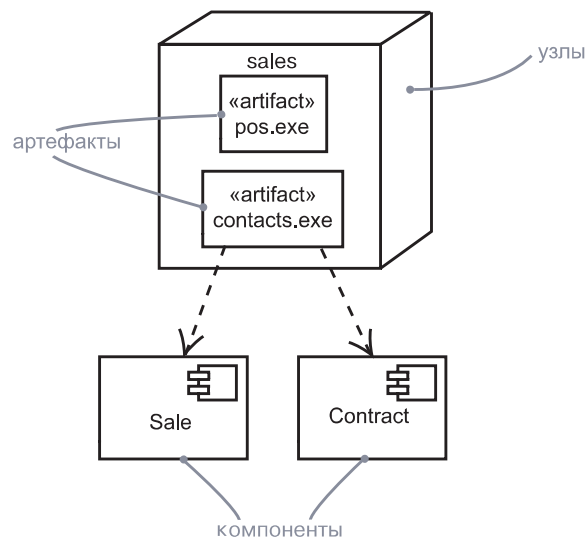


Рис. 27.3. Узлы и артефакты

Множество объектов или артефактов, приписанных к узлу как группа, называется *элементом распределения* (distribution unit).

На заметку. Узлы подобны классам в том отношении, что для них можно задать атрибуты и операции. Например, можно указать, что у узла есть атрибуты *скоростьПроцессора* и *память*, а также операции *включить*, *выключить*, *приостановить*.

Пакеты
обсуждаются
в главе 12.
Связи обсуждаются
в главе 5 и 10.

Организация узлов

Узлы можно группировать в пакеты точно так же, как классы и артефакты.

Можно также организовывать узлы, специфицируя связи зависимости, обобщения и ассоциации (включая агрегации) между ними.

Соединения

Самый распространенный вид связи между узлами — это *ассоциация*. В данном контексте ассоциация представляет физическое соединение узлов, например линию Ethernet, последовательный канал или разделяемую шину (см. рис. 27.4). Ассоциации можно использовать даже для моделирования не прямых соединений типа спутниковой линии связи между двумя удаленными процессорами.

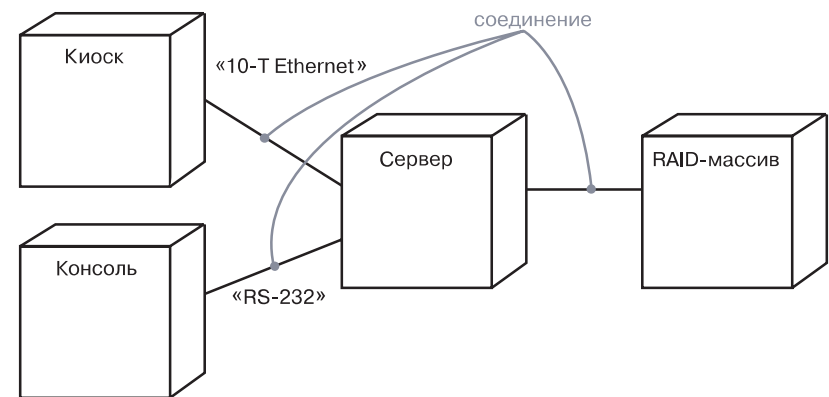


Рис. 27.4. Соединения

Поскольку узлы аналогичны классам, в нашем распоряжении находится весь аппарат ассоциаций. Иными словами, можно использовать роли, множественность и ограничения. Как показано на рис. 27.4, следует применять стереотипы, если необходимо моделировать разные виды соединений — к примеру, чтобы отличить соединение 10-T Ethernet от соединения по последовательному каналу RS-232.

Типичные приемы моделирования

Моделирование процессоров и устройств

Моделирование процессоров и устройств, образующих топологию автономной, встроенной, клиент-серверной или распределенной системы, – вот самый распространенный пример использования узлов.

Механизмы расширения UML обсуждаются в главе 6.

Поскольку все механизмы расширения UML применимы и к узлам, то для описания новых видов узлов, представляющих конкретные процессоры и устройства, часто используются стереотипы. *Процессор* (processor) – это узел, способный обрабатывать данные, то есть исполнять артефакт. *Устройство* (device) – это узел, не способный обрабатывать данные (по крайней мере, на выбранном уровне абстракции) и в общем случае используемый для представления чего-либо связанного с реальным миром.

Для моделирования процессоров и устройств вам понадобятся:

- ❑ Идентифицировать вычислительные элементы представления системы с точки зрения размещения и смоделировать каждый из них как узел.
- ❑ Если эти элементы представляют процессоры и устройства общего вида, приписать им соответствующие стандартные стереотипы. Если же это процессоры и устройства, входящие в словарь предметной области, – сопоставить им подходящие стереотипы с пиктограммой.
- ❑ Как и в случае моделирования классов, рассмотреть атрибуты и операции, применимые к каждому узлу.

В качестве примера на рис. 27.5 изображена диаграмма, которую мы рассматривали выше. Теперь каждому узлу на ней приписан стереотип. Здесь server (сервер) – это узел со стереотипом процессора общего вида; kiosk (киоск) и console (консоль) – узлы со стереотипами специализированных процессоров, а RAID farm (RAID-массив) – узел со стереотипом специализированного устройства.

На заметку. Узлы – это, возможно, именно те строительные блоки UML, которым стереотипы приписываются чаще всего. Когда в ходе проектирования программной системы вы моделируете ее с точки зрения размещения, очень важно представлять потенциальным читателям визуальную информацию. Моделируя процессор, являющийся компьютером общего назначения, присвойте ему пиктограмму компьютера. Моделируя какое-либо устройство, например сотовый телефон, факс, модем или видеокамеру, обозначьте и его подходящей пиктограммой.

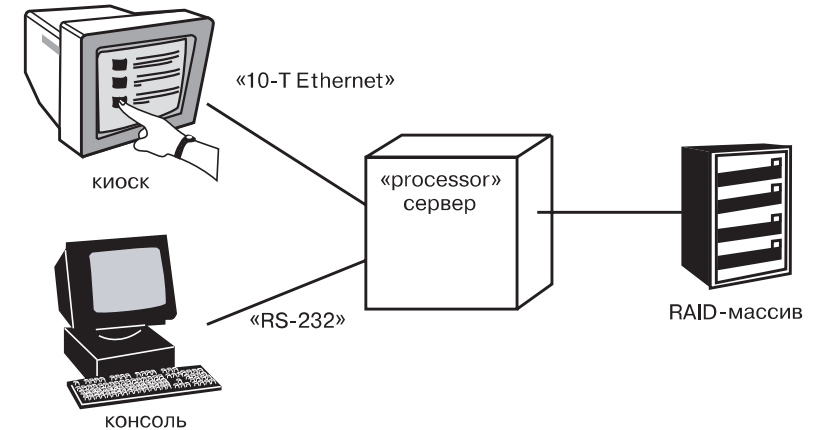


Рис. 27.5. Процессоры и устройства

Моделирование распределения артефактов

Семантика местоположения рассматривается в главе 24.

При моделировании топологии системы бывает полезно визуализировать или специфицировать физическое распределение ее артефактов по процессорам и устройствам, входящим в состав системы.

Моделирование распределения артефактов состоит из следующих шагов:

- ❑ Припишите каждый значимый компонент системы к определенному узлу.
- ❑ Рассмотрите возможности дублирования размещения артефактов. Довольно распространен случай, когда одни и те же артефакты (например, некоторые исполняемые программы и библиотеки) размещаются одновременно в нескольких узлах.
- ❑ Изобразите распределение артефактов по узлам одним из трех способов:
 1. Не делайте размещение видимым, но оставьте его на заднем плане модели, то есть в спецификации каждого узла;
 2. Соедините каждый узел с артефактами, которые на нем размещены, связью зависимости;
 3. Перечислите артефакты, размещенные на узле, в дополнительном разделе.

Экземпляры обсуждаются в главе 13, диаграммы объектов – в главе 14.

Последний из перечисленных способов проиллюстрирован на рис. 27.6, основанном на предыдущих диаграммах. Здесь специфицированы исполняемые артефакты, размещенные в каждом узле. Эта диаграмма несколько отличается от предыдущих – она является диаграммой объектов, на которой визуализированы конкретные экземпляры каждого узла. В данном случае экземпляры RAID farm

(RAID-массив) и kiosk (киоск) анонимны, а у остальных двух экземпляров есть имена: c для console (консоль) и s для server (сервер). Для каждого процессора на рисунке отведен дополнительный раздел, показывающий, какие артефакты на нем развернуты. Объект server также изображен со своими атрибутами: processorSpeed (скорость Процессора) и memory (память), причем их значения видимы. Раздел размещения может содержать список имен артефактов в текстовом виде или вложенные графические символы артефактов.

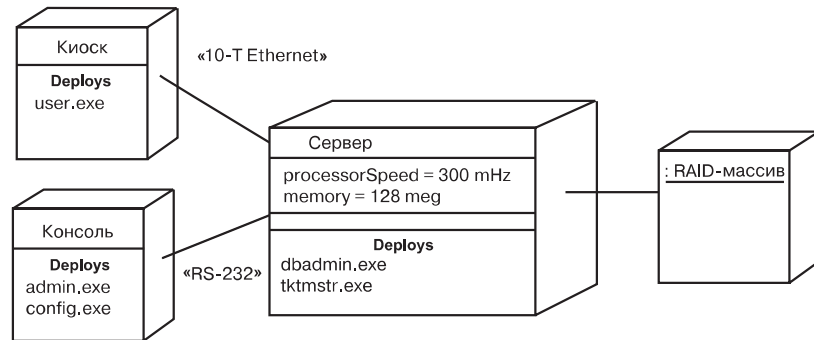


Рис. 27.6. Моделирование распределения артефактов

Советы и подсказки

Хорошо структурированный узел обладает следующими свойствами:

- ☐ представляет четкую абстракцию некоторой сущности из словаря аппаратных средств области решения;
- ☐ декомпозирован лишь до такого уровня, который необходим, чтобы выразить ваши намерения;
- ☐ раскрывает только те атрибуты и операции, которые относятся к моделируемой области;
- ☐ явно показывает набор артефактов, которые на нем расположены;
- ☐ соединен с другими узлами способом, отражающим топологию реальной системы.

Изображая узел в UML, руководствуйтесь следующими принципами:

- ☐ определите для своего проекта или организации в целом набор стереотипов с подходящими пиктограммами, которые несут очевидную для читателя смысловую нагрузку;
- ☐ показывайте только те атрибуты и операции (если таковые существуют), которые необходимы для понимания назначения узла в данном контексте.

Глава 28. Кооперации

В этой главе:

- Кооперации, реализации и взаимодействия
- Моделирование реализации варианта использования
- Моделирование реализации операции
- Моделирование механизма
- Материализация взаимодействий

В контексте системной архитектуры кооперация позволяет выделить концептуальную часть, которая подчеркивает и статические, и динамические аспекты. Кооперацией называется сообщество классов, интерфейсов и других элементов, которые работают совместно, обеспечивая поведение, представляющее собой нечто большее, чем поведение всех тех же частей в сумме.

Кооперация помогает специфицировать реализацию вариантов использования и операций, а также моделировать архитектурно значимые механизмы системы.

Введение

Представьте самое прекрасное здание, которое вы когда-либо видели, — скажем, Тадж Махал или Собор Парижской Богоматери. Красота обеих построек не поддается описанию. Их архитектура в общем-то проста, хотя заложенные в ней идеи отнюдь не лежат на поверхности. В каждой конструкции легко проглядывается симметрия. Если посмотреть внимательнее, можно заметить подробности, прекрасные сами по себе, которые в сочетании друг с другом создают впечатление гармонии и функциональности — большей, чем таится в каждой отдельной детали.

А теперь вспомните самое нелепое и отвратительное здание из виденных вами — допустим, кафе по соседству с вашим домом. Здесь налицо смешение архитектурных стилей: в общий модернистский дизайн никак не вписывается острроверхая крыша, украшения не гармонируют ни с тем, ни с другим, кричащие цвета оскорбляют взгляд. Обычно такие здания — «шедевры» обычных

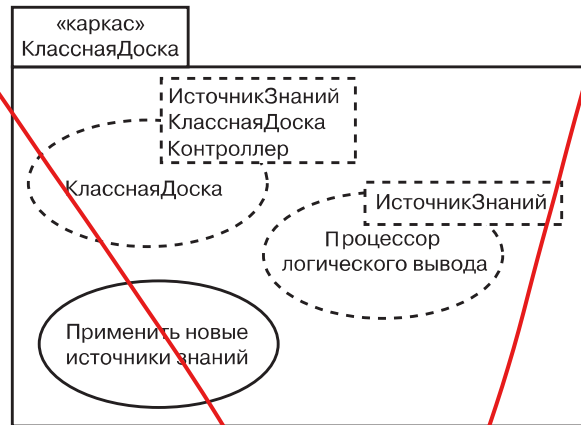


Рис. 29.7. Моделирование архитектурного образца

На заметку. На практике полное моделирование каркаса – задача, по своей сложности сопоставимая с моделированием архитектуры всей системы. В некоторых отношениях она даже сложнее, поскольку для того, чтобы с каркасом можно было работать, вы должны раскрыть все его элементы управления и стыковки и, возможно, представить метаварианты использования (типа *Apply new knowledge sources*), которые показывают, как настраивается каркас, а также простые варианты использования, поясняющие его поведение.

Советы и подсказки

При моделировании образцов в UML следует помнить, что они работают на многих уровнях абстракции, начиная от отдельных классов и заканчивая системой в целом. Самые интересные виды образцов – это механизмы и каркасы. Хорошо структурированный образец обладает следующими свойствами:

- ☐ решает типичную проблему типичным образом;
- ☐ включает структурную и поведенческую составляющие;
- ☐ раскрывает элементы управления и стыковки, с помощью которых его можно настроить на разные контексты;
- ☐ охватывает различные индивидуальные абстракции в системе.

Изображая образец в UML, необходимо:

- ☐ раскрывать те его элементы, которые следует адаптировать для применения в конкретном контексте;
- ☐ приводить варианты использования образца, а также способы его адаптации.

Глава 30. Диаграммы артефактов

В этой главе:

- Моделирование исходного кода
- Моделирование исполняемых версий
- Моделирование физических баз данных
- Моделирование адаптируемых систем
- Прямое и обратное проектирование

Диаграммы размещения – второй вид диаграмм, используемых для моделирования физических аспектов объектно-ориентированных систем, – обсуждаются в главе 31.

Диаграммы артефактов – это один из двух видов диаграмм, предназначенных для моделирования физических аспектов объектно-ориентированных систем. Диаграмма артефактов показывает организацию и зависимости между наборами артефактов.

Такие диаграммы используются для моделирования статического представления реализации системы, включая моделирование физических сущностей, размещаемых на узлах (например, исполнимых программ, библиотек, таблиц, файлов, документов разного рода). Диаграммы артефактов – это, по сути, диаграммы классов, которые сосредоточены на артефактах системы.

Диаграммы артефактов важны не только для визуализации, специфицирования и документирования систем, основанных на артефактах, но также для конструирования исполняемых систем посредством прямого и обратного проектирования.

Введение

Строительство дома не ограничивается созданием комплекта чертежей. Они, конечно, очень важны, так как помогают визуализировать, специфицировать и документировать, какой именно дом вы собираетесь построить, и обеспечить выполнение замысла с соблюдением сроков и сметы. Но рано или поздно поэтажные планы и разрезы придется воплощать в реальном каркасе из реальных материалов – дерева, камня, металла. При этом вы, скорее всего, будете использовать и уже готовые артефакты, например встроенные шкафы, окна, двери и вентиляционные решетки. А если вы

переоборудуете здание, то число готовых артефактов возрастет, – это будут целые комнаты и инженерные конструкции.

То же самое касается программного обеспечения. Для того чтобы можно было рассуждать о желаемом поведении системы, создаются диаграммы вариантов использования. Словарь предметной области описывается диаграммами классов. Чтобы стало ясно, как сущности из этого словаря совместно работают для обеспечения нужного поведения, применяются диаграммы последовательности, коммуникации, состояний и деятельности. В конечном счете логические чертежи превращаются в реалии из мира битов – исполняемые программы, библиотеки, таблицы, файлы и различные документы. При этом обнаруживается, что некоторые из этих артефактов приходится создавать «с нуля», но находят и способы повторного использования старых артефактов.

В UML диаграммы артефактов используются в целях визуализации статического аспекта физических артефактов и их связей, а кроме того, описания их деталей для конструирования, как показано на рис. 30.1.

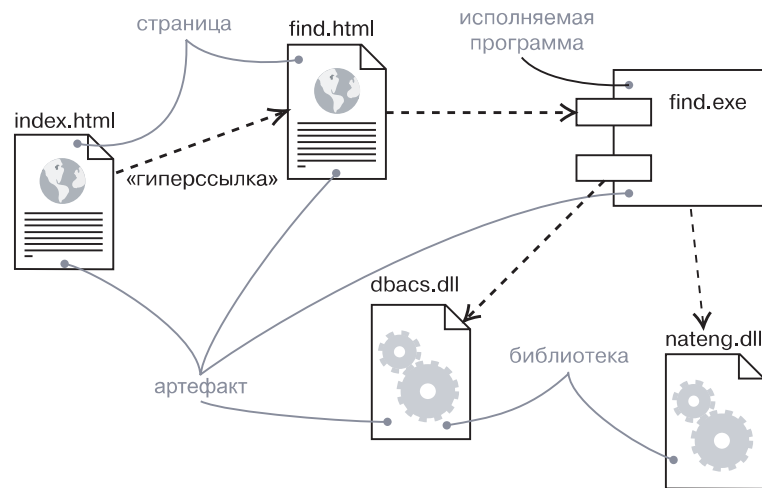


Рис. 30.1. Диаграмма артефактов

Термины и понятия

Диаграмма артефактов показывает набор артефактов и связей между ними. Изображается в виде графа с вершинами и дугами (ребрами).

Общие свойства

Общие свойства всех диаграмм обсуждаются в главе 7.

Артефакты обсуждаются в главе 26, интерфейсы –

в главе 11, связи – в главах 5 и 10, пакеты – в главе 12, подсистемы – в главе 32, экземпляры – в главе 13, диаграммы классов – в главе 8, представления реализации в контексте системной архитектуры – в главе 2.

У диаграммы артефактов, как и у всех диаграмм, есть имя и графическое наполнение. От остальных типов диаграмм она отличается своим содержимым.

Содержимое

Диаграммы артефактов обычно содержат артефакты, а также связи зависимости, обобщения, ассоциации и реализации. Подобно другим диаграммам, могут содержать примечания и ограничения.

Общее применение

Диаграммы артефактов применяются для моделирования статического представления реализации системы. Это представление в первую очередь поддерживает управление конфигурацией частей системы, состоящих из артефактов, которые могут быть собраны разными способами для построения работающей системы.

Когда моделируется статическое представление реализации системы, то диаграммы артефактов обычно используются одним из следующих четырех способов:

1. *Для моделирования исходного кода.* В большинстве современных объектно-ориентированных языков программирования код пишется в интегрированных средах разработки, сохраняющих исходные тексты в файлах. Диаграммы артефактов можно применять для моделирования конфигурации этих файлов, которые представляют собой рабочие продукты, и установки вашей системы управления конфигурацией.
2. *Для моделирования исполняемых версий.* *Версия (release)* – это относительно полный и согласованный набор артефактов, поставляемый внутреннему или внешнему пользователю. Версия в данном понимании сосредоточена на тех частях, которые необходимы для поставки работающей системы. При моделировании версий с помощью диаграмм артефактов происходят визуализация, специфицирование и документирование решений, принятых относительно физических составляющих системы, то есть артефактов размещения.

Свойство сохраняемости, или устойчивости (persistence), обсуждается в главе 24, моделирование логических схем баз данных – в главе 8.

3. *Для моделирования физических баз данных.* Представьте себе физическую базу данных как конкретную реализацию схемы, существующую в мире битов. Схемы, по сути, описывают API для доступа к хранимой информации; модель же физической базы представляет способы хранения информации в таблицах реляционной базы или на страницах объектно-ориентированной базы данных. Для представления этих и иных физических баз данных можно использовать диаграммы артефактов.
4. *Для моделирования адаптируемых систем.* Некоторые системы достаточно статичны: их компоненты появляются на сцене, принимают участие в исполнении, а затем покидают ее. Другие системы более динамичны: они включают в себя мобильных агентов, или артефакты, мигрирующие с целью балансирования нагрузки и восстановления после сбоев. Поведение таких систем моделируется диаграммами артефактов совместно с некоторыми другими диаграммами UML.

Типичные приемы моделирования

Моделирование исходного кода

При разработке программ на языке Java исходный код обычно сохраняется в файлах с расширением .java. Программы, написанные на C++, обычно хранят исходный код в заголовочных файлах с расширением .h и файлах реализации с расширением .cpp. При использовании языка IDL для разработки приложений COM+ или CORBA, единственный, с точки зрения проектирования, интерфейс распадается на четыре исходных файла: сам интерфейс, клиентский заместитель (proxy), серверную заглушку (stub) и класс-мост (bridge). По мере роста объема приложения, на каком бы языке оно ни было написано, эти файлы приходится организовывать в группы. Затем, на стадии сборки приложения, вы, вероятно, станете создавать различные варианты одних и тех же файлов для каждой новой промежуточной версии и захотите поместить все это в систему управления конфигурацией.

В большинстве случаев нет необходимости моделировать данный аспект системы непосредственно. Вместо этого вы позволите среде разработки отслеживать эти файлы и их связи. Иногда, однако, бесполезно визуализировать исходные файлы и связи между ними с помощью диаграмм артефактов. Применяемые таким образом диаграммы артефактов обычно содержат только артефакты – рабочие продукты со стереотипами файлов – вместе

Стереотип файла для артефактов обсуждается в главе 26.

с их зависимостями. Например, вы могли бы выполнить обратное проектирование набора исходных файлов для визуализации сложной системы зависимостей между ними при компиляции. Можно пойти и в другом направлении, специфицировав связи между исходными файлами и затем передав эту модель на вход инструменту компиляции, такому как программа make в UNIX. Аналогично можно использовать диаграммы артефактов для визуализации истории набора исходных файлов, хранящихся в системе управления конфигурацией. Получая из этой системы информацию (например, о том, сколько раз некий файл извлекался для редактирования на протяжении определенного периода времени), вы можете использовать ее для «раскраски» диаграммы артефактов, что поможет выявить среди исходных файлов «горячие точки», в которых архитектура системы чаще всего подвергается модификациям.

Чтобы смоделировать исходный код системы, необходимо:

- Применяя прямое или обратное проектирование, идентифицировать набор представляющих интерес файлов исходного кода и смоделировать их как артефакты со стереотипом file.
- Для крупных систем использовать пакеты, чтобы показать группы исходных файлов.
- Рассмотреть возможность показа с помощью помеченных значений такой информации, как номер версии файла, его автор, дата последнего изменения. Для управления этими значениями применять инструментальные средства.
- Смоделировать зависимости компиляции между исходными файлами с помощью связей зависимости. Опять же, для того чтобы сгенерировать эти зависимости и управлять ими, понадобятся инструментальные средства.

Стереотип зависимости trace обсуждается в главе 10.

В качестве примера на рис. 30.2 показаны пять файлов исходного кода. Файл signal.h – заголовочный. Представлены три его версии, начиная с последней и заканчивая первой. Каждая версия помечена значением, указывающим ее номер.

Заголовочный файл signal.h используется двумя другими – interp.cpp и signal.cpp. Первый из них при компиляции зависит от другого заголовочного файла, irq.h. В свою очередь, файл device.cpp зависит от interp.cpp. Имея перед глазами такую диаграмму, легко проследить, что произойдет в случае изменений. В частности, изменение исходного файла signal.h потребует перекомпиляции трех других файлов – signal.cpp, interp.cpp и, как следствие, device.cpp. Из той же диаграммы видно, что irq.h не коснутся преобразования.

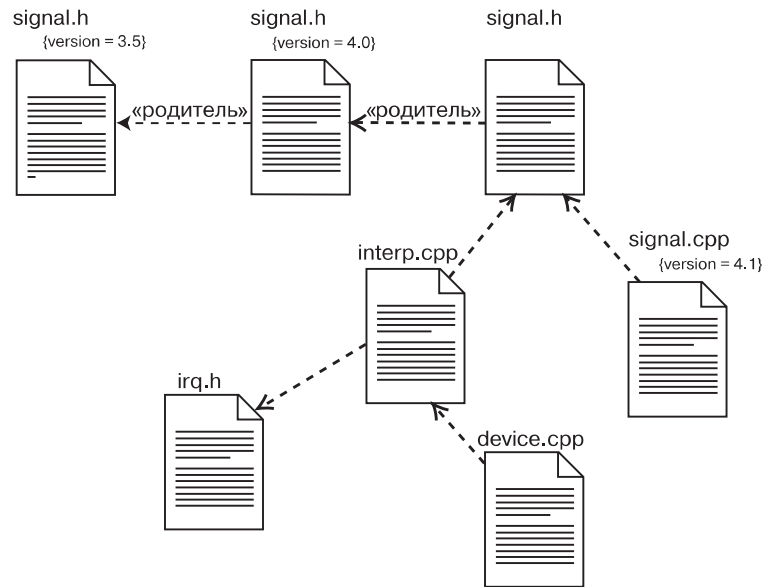


Рис. 30.2. Моделирование исходного кода

Подобные диаграммы легко генерируются путем обратного проектирования на основе информации, хранящейся в системе управления конфигурацией среды разработки.

Моделирование исполняемой версии

Выпуск версий простого приложения не составляет труда — единственная исполняемая программа сбрасывается на диск, и пользователи просто запускают ее. Для подобных приложений диаграммы артефактов не требуются: там нечего визуализировать, специфицировать, конструировать и документировать.

Выпуск версий более сложных приложений все-таки предполагает трудозатраты, иногда немалые. Кроме главной исполняемой программы (обычно файла с расширением .exe), нужен ряд вспомогательных модулей (обычно это файлы с расширением .dll, если вы работаете в контексте COM+, либо .class или .jar — при работе с языком Java), баз данных, файлов подсказки и ресурсных файлов. Для распределенных систем, вероятно, понадобится несколько исполняемых программ и других файлов, разбросанных по разным узлам. Если вы работаете с системой приложений, одни артефакты могут оказаться уникальными, а другие будут использоваться в нескольких приложениях. По мере развития системы управление конфигурацией всего этого множества артефактов приобретает все

большую важность и вместе с тем усложняется, поскольку изменение некоторых артефактов в одном приложении может отразиться на работе других.

По этой причине для визуализации, специфицирования, конструирования и документирования исполняемых версий (включая артефакты размещения, формирующие каждую версию, и связи между этими артефактами) используются диаграммы артефактов. Вы можете применять их для прямого проектирования новой системы и для обратного проектирования существующей.

Создавая диаграммы артефактов, подобные той, что изображена на рис. 30.2, вы на самом деле просто моделируете часть сущностей и связей, которые представляют реализацию вашей системы. По этой причине каждая диаграмма артефактов должна сосредотачивать внимание только на одном текущем наборе артефактов.

Чтобы смоделировать исполняемую версию, необходимо:

- ❑ Идентифицировать набор артефактов, подлежащих моделированию. Обычно это выборка либо вся совокупность артефактов, размещенных на одном узле, или же ряд артефактов, распределенных по узлам системы.
- ❑ Рассмотреть стереотипы каждого артефакта в наборе. Для большинства систем обнаружится ограниченное число разных видов артефактов (таких как исполняемые программы, библиотеки, таблицы, файлы и документы). Для визуального представления этих стереотипов можно воспользоваться механизмами расширения UML.
- ❑ Для каждого артефакта в наборе рассмотреть его связь с окружающими его артефактами. Чаще всего это интерфейсы, которые экспортирует (реализует) один артефакт и импортируют (используют) другие. Если нужно раскрыть соединения в системе, — явно смоделировать эти интерфейсы. Если же необходимо представить модель на более высоком уровне абстракции, раскрыть эти связи, показывая только зависимости между артефактами.

В качестве примера на рис. 30.3 представлена модель части исполняемой версии автономного робота. Основное внимание обращено на артефакты размещения, ассоциированные с функциями перемещения робота и вычислительными операциями. Вы видите один артефакт — driver.dll, который материализует компонент Driving (Перемещение), экспортирующий интерфейс IDrive (), импортируемый компонентом Path (Путь), материализуемым другим артефактом, path.dll. Зависимость между компонентами Path и Driving указывает на зависимость между артефактами path.dll и driver.dll, которые реализуют их. На диаграмме присутствует еще

Механизмы расширения UML обсуждаются в главе 6, интерфейсы — в главе 11.

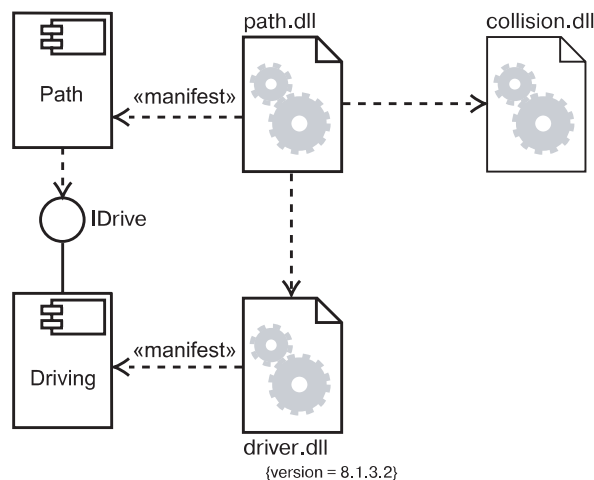


Рис. 30.3. Моделирование исполняемой версии

один артефакт — `collision.dll`, который также материализует компонент, хотя эти подробности и скрыты (показана лишь прямая зависимость `path.dll` от `collision.dll`).

В данной системе участвует множество других артефактов. Однако диаграмма сосредоточена на тех артефактах размещения, которые непосредственно относятся к процессу движения робота. Заметьте, что при такой компонентной архитектуре вы могли бы заменить конкретную версию `driver.dll` на другую — при условии, что она реализует те же (и, возможно, какие-то дополнительные) интерфейсы, — и `path.dll` мог бы по-прежнему правильно работать.

Моделирование физической базы данных

Моделирование схемы логической базы данных обсуждается в главе 8.

Логическая схема базы данных охватывает словарь хранимых данных системы вместе с семантикой их связей. Физически все эти сущности сохраняются в базе данных — либо в реляционной, либо в объектно-ориентированной, либо в гибридной (объектно-реляционной) — для последующего извлечения. UML так же хорошо приспособлен к моделированию физических баз данных, как и к моделированию их логических схем.

Отображение логической схемы базы данных на объектно-ориентированную базу достаточно прямолинейно, поскольку даже сложные цепочки наследования могут быть сохранены без какого-либо преобразования. Однако отобразить логическую схему на реляционную базу не так просто. Если имеет место наследование, приходится принимать решения относительно того, как отображать

Проектирование физических баз данных не входит в круг тем, рассматриваемых в этой книге, — здесь мы сосредотачиваем внимание на показе отдельных приемов моделирования баз данных и таблиц в UML.

классы на таблицы. Обычно применяется либо одна из трех ниже-названных стратегий, либо их комбинация:

1. *Спуск (push down)* — определяется отдельная таблица для каждого класса. Это простой, но примитивный подход, поскольку он вызывает проблемы сопровождения при добавлении новых дочерних классов или модификации существующих родительских;
2. *Подъем (pull up)* — все цепочки наследования свертываются так, что реализации любого класса в иерархии имеют одинаковое состояние. Недосток в том, что во многих экземплярах приходится хранить избыточную информацию;
3. *Расщепление таблиц (split tables)* — данные родительского и дочернего класса разносятся по разным таблицам. Такой подход лучше отображает структуру наследования, но его недостаток состоит в том, что для доступа к данным приходится соединять многие таблицы.

Проектируя физическую базу данных, необходимо также решить, как следует отображать операции, определенные в логической схеме. Объектно-ориентированные базы данных обеспечивают достаточно прозрачное отображение, но что касается реляционных, здесь приходится решать, как реализовать эти логические операции. Опять же, существует несколько вариантов:

1. Простые операции создания, выборки, обновления и удаления реализуются стандартными вызовами SQL или ODBC;
2. Более сложное поведение (такое как бизнес-правила) отображается на триггеры и хранимые процедуры.

С учетом приведенных соображений для моделирования физической базы данных необходимо:

- ❑ Идентифицировать присутствующие в модели классы, которые представляют логическую схему базы данных.
- ❑ Выбрать стратегию отображения этих классов на таблицы. Следует рассмотреть и физическое распределение баз данных. Необходимо проанализировать физическое размещение данных в системе — от этого тоже зависит выбор стратегии.
- ❑ Создать диаграмму артефактов для визуализации, специфирования, конструирования и документирования отображения, в которой артефакты имеют стереотип таблицы.
- ❑ По возможности использовать инструментальные средства для преобразования логической схемы в физическую.

На рис. 30.4 показан набор таблиц базы данных, взятых из информационной системы учебного заведения. Вы видите одну базу — `school.db`, изображенную в виде артефакта со стереотипом `database`.

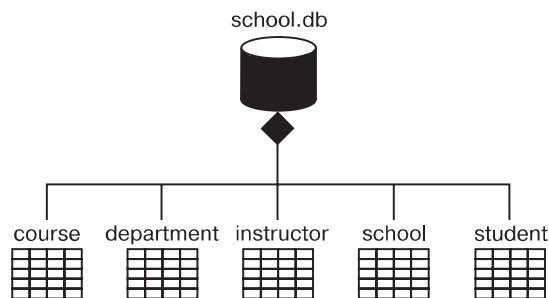


Рис. 30.4. Моделирование физической базы данных

Она состоит из пяти таблиц: student (студент), class (класс), instructor (инструктор), department (отдел) и course (курс). В соответствующей логической схеме базы данных нет наследования, поэтому отобразить ее на физическую несложно.

Хотя в данном примере это не показано, вы можете специфицировать содержимое каждой таблицы. Артефакты могут иметь атрибуты, поэтому общая идиома при моделировании физической базы данных заключается в использовании атрибутов для специфирования столбцов каждой таблицы.

Аналогичным образом артефакты могут иметь операции, которые позволяют отображать хранимые процедуры.

Моделирование адаптируемых систем

Все диаграммы артефактов, показанные до сих пор, использовались для моделирования статических представлений. Их артефакты проводили всю свою жизнь на одном узле. Хотя эта ситуация встречается чаще всего, иногда, особенно при работе со сложными распределенными системами, все же приходится моделировать и динамические представления. Например, можно представить систему, которая реплицирует свои базы данных на несколько узлов, переключаясь на резервный сервер в случае отказа главного. Аналогично при моделировании глобальной распределенной системы, работающей в режиме 24x7 (то есть 7 дней в неделю, 24 часа в сутки), вы, скорее всего, столкнетесь с мобильными агентами — артефактами, которые мигрируют с одного узла на другой для обслуживания некоторой транзакции. Чтобы смоделировать такие динамические представления, вам понадобится использовать комбинацию диаграмм артефактов, объектов и взаимодействия.

Для моделирования адаптируемой системы необходимо:

- Рассмотреть физическое распределение артефактов, которые могут мигрировать с одного узла на другой. Вы можете

Атрибут

location

обсуждается в главе 24, диаграммы объектов — в главе 14.

специфицировать местоположение экземпляра артефакта, пометив его атрибутом location, который можно отобразить на диаграмме артефактов.

- Если нужно смоделировать действия, вызывающие миграцию артефакта, создать с этой целью соответствующую диаграмму взаимодействия, которая будет содержать экземпляры артефактов. Изменение местоположения артефакта можно проиллюстрировать, нарисовав один экземпляр несколько раз, но с разными значениями состояния, включая его местоположение.

На рис. 30.5 представлено моделирование репликации базы данных, уже рассматривавшейся выше (см. рис. 30.4). Здесь мы видим два экземпляра артефакта school.db. Оба анонимны и имеют разные помеченные значения местоположения (location). Также присутствует примечание, которое явно указывает, какой экземпляр куда реплицируется.

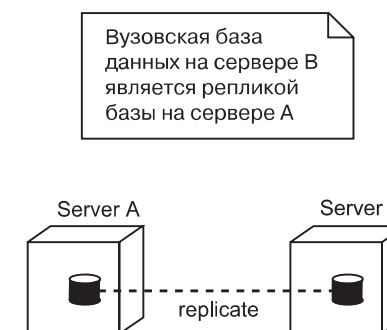


Рис. 30.5. Моделирование адаптируемых систем

Если необходимо показать подробности каждой базы данных, их можно изобразить в канонической форме — в виде артефакта со стереотипом database.

Хотя на этом рисунке это не показано, можно использовать и диаграмму взаимодействия для моделирования динамики переключения с главной базы данных на резервную.

Диаграммы взаимодействия обсуждаются в главе 19.

Прямое и обратное проектирование

Прямое и обратное проектирование артефактов довольно просто, потому что артефакты сами по себе — физические сущности (исполняемые программы, библиотеки, таблицы, файлы и разнообразные документы), а потому близки к работающей системе. При прямом проектировании класса или кооперации вы на самом

деле проектируете артефакт, который представляет исходный код, двоичную библиотеку или исполняемую программу для этого класса или кооперации. Точно так же при обратном проектировании исходного кода, двоичной библиотеки или исполняемой программы вы на самом деле выполняете обратное проектирование артефакта или набора артефактов, которые, в свою очередь, отображаются на классы или кооперации.

Приступая к прямому проектированию (созданию кода из модели) класса или кооперации, вы должны решить, во что нужно их преобразовывать: в исходный код, двоичную библиотеку или исполняемую программу. Логическую модель имеет смысл превратить в исходный код, если вас интересует управление конфигурацией файлов, которые затем будут переданы среде разработки. Если же вас интересует управление артефактами, которые фактически будут развернуты в составе работающей системы, то логические модели следует непосредственно преобразовать в двоичные библиотеки или исполняемые программы. Иногда нужно и то, и другое. Классу или кооперации можно поставить в соответствие как исходный код, так и двоичную библиотеку или исполняемую программу.

Чтобы выполнить прямое проектирование диаграммы артефактов, необходимо:

- ❑ Идентифицировать классы и кооперации, которые реализует каждый артефакт. Выразить эту реализацию при помощи связи материализации.
- ❑ Выбрать форму представления каждого артефакта. Это может быть либо исходный код (форма, поддающаяся управлению средствами разработки), либо двоичная библиотека или исполняемая программа (форма, которая может быть включена в работающую систему).
- ❑ Использовать инструментальные средства для прямого проектирования модели.

Обратное проектирование диаграмм классов обсуждается в главе 8.

Обратное проектирование диаграммы артефактов (создание модели из кода) — не идеальный процесс, поскольку при этом всегда происходит потеря информации. По исходному коду можно обратно спроектировать классы — это в общем-то тривиальная задача. Обратное проектирование артефактов из исходного кода выявляет существующие между файлами зависимости компиляции. Если же говорить о двоичных библиотеках, то самое большее, на что можно рассчитывать, — обозначить библиотеку как артефакт, а затем, используя обратное проектирование, раскрыть его интерфейсы. Это второе по распространенности действие, которое выполняется над диаграммами артефактов. Такой подход может быть полезен при ознакомлении с новыми плохо документированными

библиотеками. Максимум, что можно сделать в отношении исполняемой программы, — обозначить ее как артефакт, а затем дизассемблировать ее код, но вряд ли стоит этим заниматься, если только вы не пишете на языке ассемблера.

Чтобы выполнить обратное проектирование диаграммы артефактов, необходимо:

- ❑ Выбрать целевое представление. Исходный код можно реконструировать в артефакты, а затем в классы. Двоичные библиотеки можно подвергнуть обратному проектированию, чтобы раскрыть их интерфейсы. Исполняемые программы поддаются обратному проектированию в наименьшей степени.
- ❑ Используя инструментальные средства, указать код, который следует подвергнуть обратному проектированию. Использовать инструменты для генерации новой модели или модификации существующей, для которой ранее было выполнено прямое проектирование.
- ❑ Опрашивая модель и используя инструментальное средство, создать диаграмму артефактов — например, начать с одного или нескольких артефактов, а затем расширить диаграмму, следуя по связям или переходя к соседним артефактам. Отображать или скрывать детали этой диаграммы артефактов в соответствии с вашими установками.

В качестве примера на рис. 30.6 показана диаграмма артефактов, которая представляет результат обратного проектирования артефакта ActiveX `vbrun.dll`. Видно, что он реализует 11 интерфейсов.

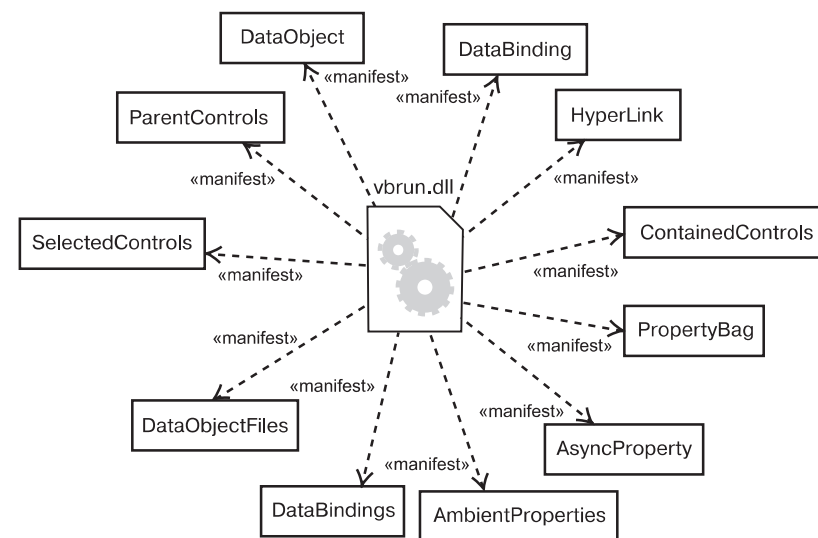


Рис. 30.6. Обратное проектирование диаграммы артефактов

Имея перед глазами такую диаграмму, можно понять семантику артефакта, перейдя к раскрытию его интерфейсных классов.

Чаше всего при обратном проектировании исходного кода, а иногда и двоичных библиотек и исполняемых программ, прибегают к помощи системы управления конфигурацией. Это означает, что вы будете работать с конкретными версиями файлов и библиотек, совместимых друг с другом. В таких случаях полезно включать помеченное значение, указывающее версию артефакта, – ее может предоставить система управления конфигурацией. Таким образом, можно использовать UML для визуализации истории артефакта при смене версий системы.

Советы и подсказки

При создании диаграмм артефактов на UML следует помнить о том, что каждая такая диаграмма – это графическое представление статического представления реализации системы. А это значит, что ни одна отдельная диаграмма артефактов не должна отражать все, что известно о представлении реализации системы. Все диаграммы артефактов, взятые в совокупности, представляют систему с точки зрения реализации, но каждая в отдельности описывает лишь один аспект.

Хорошо структурированная диаграмма артефактов обладает следующими свойствами:

- ❑ сосредоточена на выражении какого-то одного аспекта статического представления реализации системы;
- ❑ содержит только те элементы, которые существенны для понимания данного аспекта;
- ❑ представляет уровень детализации, соответствующий ее уровню абстракции, включая лишь те дополнения, которые важны для понимания на этом уровне;
- ❑ не настолько лаконична, чтобы неверно информировать читателя о важной семантике.

Изображая диаграмму артефактов, необходимо руководствоваться следующими принципами:

- ❑ присваивать диаграмме имя, описывающее ее назначение;
- ❑ располагать элементы так, чтобы минимизировать количество пересекающихся линий;
- ❑ пространственно организовывать элементы так, чтобы семантически близкие сущности оказывались рядом;
- ❑ использовать примечания и цветовые выделения для привлечения внимания к важным моментам диаграммы;
- ❑ осторожно применять элементы со стереотипами. Выделять небольшой набор общих для проекта (или организации) пиктограмм и использовать их согласованно.

Глава 31. Диаграммы размещения

В этой главе

- Моделирование встроенных систем
- Моделирование клиент-серверных систем
- Моделирование полностью распределенных систем
- Прямое и обратное проектирование

Диаграммы артефактов – второй вид диаграмм для моделирования физических аспектов объектно-ориентированных систем – обсуждаются в главе 30.

Диаграммы размещения – это один из двух видов диаграмм, используемых при моделировании физических аспектов объектно-ориентированной системы. Такая диаграмма представляет конфигурацию узлов, где производится обработка информации, и показывает, какие артефакты размещены на каждом узле.

Диаграммы размещения используются для моделирования статического представления системы с точки зрения размещения. В основном под этим понимается моделирование топологии аппаратных средств, на которых работает система. По существу, диаграммы размещения – это просто диаграммы классов, сосредоточенные на системных узлах.

Диаграммы размещения важны не только для визуализации, специфицирования и документирования встроенных, клиент-серверных и распределенных систем, но и для управления исполняемыми системами с использованием прямого и обратного проектирования.

Введение

При создании программной системы вы как разработчик программного обеспечения обращаете внимание в первую очередь на архитектуру и размещение своих программ в вычислительной среде. Но в качестве системного инженера вы заинтересованы главным образом в аппаратных и программных средствах системы и в том, как достичь оптимального их сочетания. Иными словами, разработчики программного обеспечения имеют дело с неосязаемыми артефактами вроде модели и кода, а разработчики систем – еще и с аппаратурой, вполне осязаемой.

Хотя основное назначение языка UML – визуализация, специфицирование, конструирование и документирование программных артефактов, он применим также и для работы с аппаратными артефактами. Из этого не следует, что UML – универсальный язык описания аппаратных средств наподобие VHDL. Однако он все же способен моделировать многие аппаратные аспекты системы, чего разработчику программного обеспечения достаточно для описания платформы, на которой система будет работать, а системному инженеру – для сопряжения программных и аппаратных средств. В UML представление о структуре программной системы дают диаграммы классов и компонентов. Для специфицирования поведения программ применяются диаграммы последовательности, коммуникации, состояния и деятельности. А на стыке программной и аппаратной сфер находятся диаграммы размещения, позволяющие говорить о топологии процессоров и устройств, на которых выполняется система.

В UML диаграммы размещения используются для визуализации статических аспектов физических узлов и их взаимосвязей, а также для описания их деталей, которые имеют отношения к конструированию систем (см. рис. 31.1).

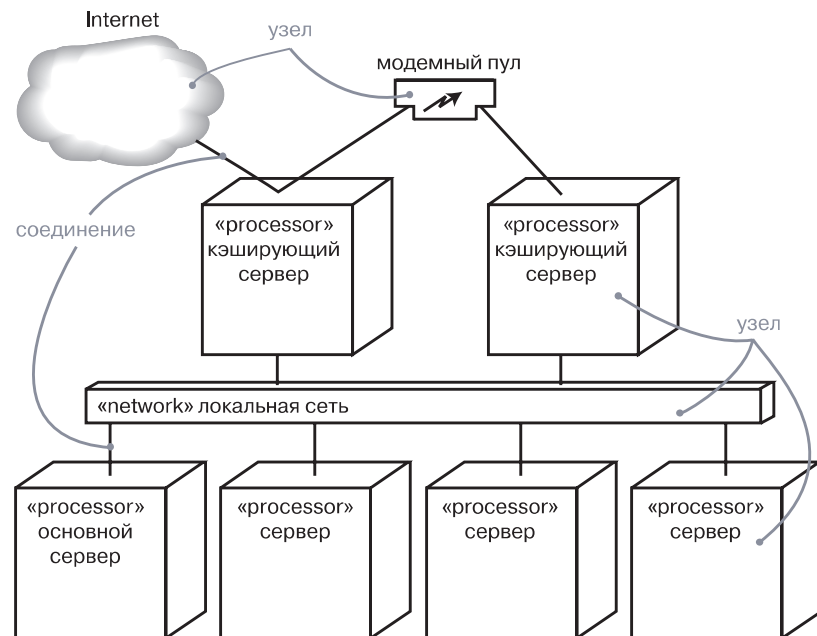


Рис. 31.1. Диаграмма размещения

Базовые понятия

На *диаграмме размещения* (deployment diagram) показана конфигурация обрабатывающих узлов и артефактов, размещенных в этих узлах. Диаграмма размещения представлена в виде графа с ребрами и вершинами.

Общие свойства

Диаграмма размещения обладает общими свойствами, присущими всем диаграммам, – именем и графическим содержанием, которое отражает одну из проекций модели. От других диаграмм отличается своим специфическим наполнением.

Содержание

Диаграммы размещения обычно включают в себя узлы, а также связи зависимости и ассоциации; подобно всем прочим диаграммам, могут содержать примечания и ограничения. На диаграммах размещения бывают представлены артефакты, каждый из которых должен располагаться на каком-нибудь узле, а кроме того, пакеты или подсистемы, – и те и другие используются для группирования элементов модели в крупные блоки. Иногда бывает полезно поместить в диаграмму объектов еще и экземпляры, особенно если вы хотите визуализировать один экземпляр из семейства топологии аппаратных средств.

На заметку. Во многих отношениях диаграмма размещения является разновидностью диаграмм классов, фокусирующей внимание прежде всего на системных узлах.

Типичное применение

Диаграммы размещения используются для моделирования статического вида системы с точки зрения размещения. Это представление в первую очередь обращено на распределение, поставку и установку частей, из которых состоит физическая система.

Есть несколько типов систем, для которых диаграммы размещения не нужны. Если вы разрабатываете программу, исполняемую на одной машине и обращающуюся только к стандартным устройствам на этой же машине, управление которыми полностью возложено на операционную систему (возьмем для примера клавиатуру, дисплей и модем персонального компьютера), то диаграммы размещения можно игнорировать. Но если разрабатываемая программа

Общие свойства диаграмм обсуждаются в главе 7.

Узлы обсуждаются в главе 27, связи – в главах 5 и 10, артефакты – в главе 26, пакеты – в главе 12, подсистемы – в главе 32, экземпляры – в главе 13, диаграммы – в главе 8.

обращается к устройствам, которыми операционная система обычно не управляет, или эта программа физически размещена на разных процессорах, то диаграмма размещения поможет выявить отношения между программными и аппаратными средствами.

При моделировании статического представления системы с точки зрения размещения диаграммы размещения используются, как правило, в трех случаях:

1. *Для моделирования встроенных систем.* Встроенной (embedded) системой называется аппаратный комплекс, взаимодействующий с физическим миром, в котором велика роль программного обеспечения. Встроенные системы управляют двигателями, приводами и дисплеями, а сами управляются внешними воздействиями, например датчиками температуры и перемещения. Диаграмму размещения можно использовать для моделирования устройств и процессоров, из которых состоит встроенная система.
2. *Моделирование клиент-серверных систем.* Клиент-серверная (client/server) система – это типичный пример архитектуры, где основное внимание уделяется четкому распределению обязанностей между интерфейсом пользователя, существующим на клиенте, и хранимыми данными системы, существующими на сервере. Клиент-серверные системы находятся на одном конце спектра распределенных систем и требуют от вас принятия решений о том, как связать клиенты и серверы сетью, а также о том, как физически распределены программные артефакты между узлами. Диаграммы размещения позволяют моделировать топологию такой системы.
3. *Моделирование полностью распределенных систем.* На другом конце спектра распределенных систем находятся те из них, которые распределены широко или даже глобально (fully distributed) и охватывают серверы различных уровней. Часто на таких системах устанавливаются разные версии программных компонентов, часть которых даже мигрирует с одного узла на другой. Проектирование подобной системы требует решений, которые допускают непрерывное изменение системной топологии. Диаграммы размещения можно использовать для визуализации текущей топологии и распределения артефактов системы, чтобы можно было осмысленно говорить о влиянии на нее различных изменений.

Типичные приемы моделирования

Моделирование встроенной системы

Устройства и узлы обсуждаются в главе 27.

Механизмы расширения UML обсуждаются в главе 6.

Разработка встроенной системы не сводится к созданию программного обеспечения: ведь приходится управлять физическим миром, где движущиеся части имеют склонность ломаться, сигналы зашумлены, а поведение нелинейно. При моделировании такой системы нужно принимать во внимание взаимодействие ее интерфейса с внешним миром, а это, как правило, нестандартные устройства и узлы.

Диаграммы размещения способны облегчить общение инженеров-электронщиков и разработчиков программного обеспечения. Используя узлы со стереотипами, пиктограммы которых напоминают знакомые устройства, можно создавать диаграммы, которые одинаково понятны и тем, и другим. Диаграммы размещения также помогают анализировать баланс между программными и аппаратными средствами. Таким образом, они применяются для визуализации, специфицирования, конструирования и документирования проектных решений в части системной инженерии.

Для моделирования встроенной системы следует:

- ❑ Идентифицировать уникальные устройства и узлы.
- ❑ Позаботиться о визуальных обозначениях нестандартных устройств, воспользовавшись механизмами расширения UML для присвоения специфическим стереотипам подходящих пиктограмм. Как минимум необходимо различать процессоры, на которых размещены программные компоненты, и устройства, которые на данном уровне абстракции не содержат таковых.
- ❑ Смоделировать отношения между процессорами и устройствами на диаграмме размещения. Кроме того, специфицировать связи между артефактами представления системы с точки зрения реализации и узлами представления системы с точки зрения размещения.
- ❑ При необходимости раскрыть описание наиболее «интеллектуальных» устройств, смоделировав их структуру на более подробной диаграмме размещения.

В качестве примера на рис. 31.2 показана аппаратная реализация автономного робота. Вы видите один узел – Pentium motherboard

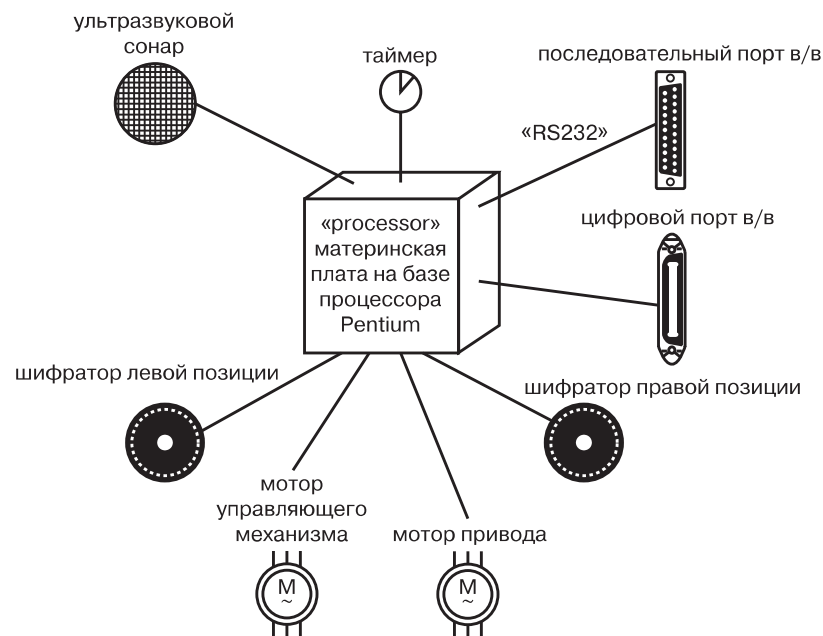


Рис. 31.2. Моделирование встроенной системы

(материнская плата на базе процессора Pentium) со стереотипом processor (процессор). Вокруг этого узла расположено восемь устройств, каждое из которых имеет стереотип device и представлено пиктограммой, похожей на реальный прототип.

Моделирование клиент-серверной системы

Приступая к разработке системы, программное обеспечение которой уже не помещается на одном процессоре, вы сразу же сталкиваетесь с целым рядом вопросов. Как оптимально распределить программные артефакты по узлам? Как они будут обмениваться информацией? Как быть с отказами и помехами? На одном конце спектра распределенных систем находятся клиент-серверные системы, в которых ясно прослеживается граница между пользовательским интерфейсом, обычно расположенном на клиенте, и данными, обычно хранящимися на сервере.

Здесь существует множество вариаций. Например, можно выбрать «тонкого» клиента, вычислительные мощности которого ограничены и который, следовательно, занят в основном взаимодействием с пользователем и отображением информации. У «тонких» клиентов может даже не быть собственных артефактов — вместо этого они загружают артефакты с сервера по мере необходимости, как, скажем, в случае с Enterprise JavaBeans. С другой стороны,

можно выбрать и «толстого» клиента, у которого вычислительных ресурсов больше и который вследствие этого может заниматься не только визуализацией. Выбор между «толстым» и «тонким» клиентом — это архитектурное решение, на которое влияют различные технические, экономические и политические факторы.

В любом случае при разделении системы на клиентскую и серверную части предстоит принять непростые решения о том, где физически разместить артефакты и как обеспечить баланс обязанностей между ними. Например, архитектура большинства систем управления информацией трехуровневая, то есть пользовательский интерфейс, бизнес-логика и база данных физически отделены друг от друга. Решение о том, куда поместить интерфейс и базу данных, как правило, очевидно, а вот понять, где должны находиться артефакты, реализующие бизнес-логику, куда сложнее.

Диаграммы размещения UML можно использовать для визуализации, специфицирования и документирования решений относительно топологии клиент-серверной системы и распределения программных артефактов между клиентом и сервером. Обычно требуется создать одну диаграмму размещения для системы в целом и ряд дополнительных диаграмм, детализирующих отдельные ее сегменты.

Для моделирования клиент-серверной системы вам понадобится:

- ☐ Идентифицировать узлы, представляющие процессоры клиента и сервера.
- ☐ Выделить те устройства, которые так или иначе влияют на поведение системы. Например, вы, скорее всего, захотите смоделировать устройство считывания кредитных карт и устройство отображения, отличные от стандартных мониторов, поскольку их расположение в топологии аппаратных средств системы имеет важное значение с точки зрения системной архитектуры.
- ☐ С помощью стереотипов разработать визуальные обозначения для процессоров и устройств.
- ☐ Смоделировать топологию узлов на диаграмме размещения. Также специфицировать связи между артефактами представления системы с точки зрения реализации и узлами представления системы с точки зрения размещения.

На рис. 31.3 показана топология системы, следующей классической клиент-серверной архитектуре. Мы видим, что граница между клиентом и сервером проведена явно путем использования пакетов client и server. Пакет client содержит два узла: console (консоль) и kiosk (киоск), имеющих свои стереотипы и потому визуально легко отличимых. Пакет server содержит два вида узлов: caching server (кэширующий сервер) и server (сервер), для каждого из которых

*Пакеты
обсуж-
даются
в главе 12,
множест-
венность
рассмат-
ривается
в главе 10.*

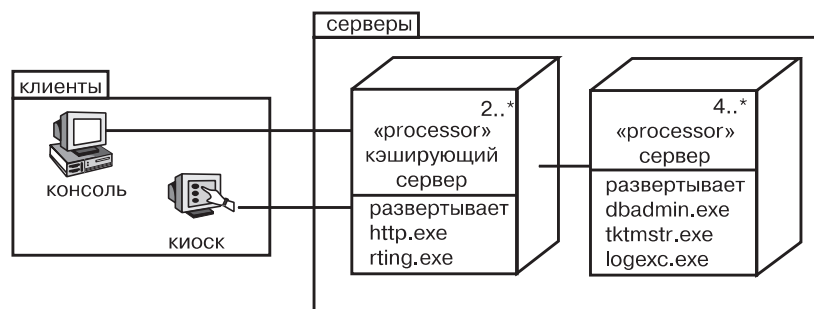


Рис. 31.3. Моделирование клиент-серверной системы

дополнительно описаны артефакты, размещаемые в узле. Заметьте, что как для caching server, так и для server указаны множественности, специфицирующие, сколько экземпляров ожидается в конкретной конфигурации. К примеру, из диаграммы видно, что кэширующих серверов должно быть не меньше двух.

Моделирование полностью распределенной системы

Распределенные системы могут быть самыми разными — от простых двухпроцессорных до разветвленных, размещенных на многих географически удаленных узлах. Последние, как правило, не бывают статическими. Узлы появляются и исчезают по мере изменения сетевого трафика и выходов процессоров из строя; создаются новые, более быстрые каналы связи, функционирующие параллельно медленным, постепенно устаревающим, которые в конце концов демонтируются. Изменяется не только топология системы, но и распределение программных артефактов. Например, таблицы баз данных могут реплицироваться между серверами с целью приблизить их к потребителю информации по мере изменения трафика. В некоторых глобальных системах артефакты могут мигрировать вслед за солнцем, перемещаясь с одного сервера на другой по мере того, как рабочий день начинается в одной части света и заканчивается в другой.

Визуализация, специфицирование и документирование топологии полностью распределенных систем представляют собой ценное подспорье для администратора, который должен вести учет вычислительных средств системы. Для этого можно применять диаграммы размещения UML. Документируя полностью распределенную систему, вы можете раскрыть детали сетевых устройств, представляя их в виде узлов со стереотипами.

Для моделирования полностью распределенной системы вам потребуется:

- ❑ Идентифицировать устройства и процессоры, как и в случае с более простой клиент-серверной системой.
- ❑ Если необходимо сделать выводы о производительности сетевой инфраструктуры или о влиянии на сеть изменений, — не забыть промоделировать коммуникационные устройства со степенью детализации, достаточной для такого рода оценок.
- ❑ Обратить особое внимание на логическое группирование узлов; для этого можно воспользоваться пакетами.
- ❑ Смоделировать устройства и процессоры с помощью диаграмм размещения. Всюду, где есть возможность, пользоваться инструментальными средствами, которые извлекают сведения о топологии системы путем обследования сети.
- ❑ Если необходимо сфокусировать внимание на динамике системы, — включить диаграммы вариантов использования для специфицирования поведения, представляющего интерес, и раскрыть варианты использования с помощью диаграмм взаимодействия.

*Пакеты
обсуж-
даются
в главе 12.*

*Варианты
использова-
ния обсуж-
даются
в главе 17,
диаграммы
взаимо-
действия —
в главе 19,
экземпля-
ры —
в главе 13.*

На заметку. При моделировании полностью распределенной системы саму сеть часто также изображают в виде узла. Например, можно представить Internet, как показано на рис. 31.1, в виде узла со стереотипом. Таким же образом позволяет оформить локальную (LAN) или глобальную (WAN) сеть — см. рис. 31.1. В любом случае вы можете воспользоваться атрибутами и операциями узла для описания свойств сети.

На рис. 31.4 показана топология полностью распределенной системы. Эта конкретная диаграмма размещения является также диаграммой объектов, поскольку содержит только экземпляры. Вы видите три консоли (анонимные экземпляры узла console со стереотипом), которые связаны с Internet (очевидно, узлом-одиночкой). С другой стороны, есть три экземпляра региональных серверов (Regional server), которые служат клиентской частью для национальных серверов (Country server), из которых показан только один. Как следует из примечания, национальные серверы соединены друг с другом, но такие связи на диаграмме не отражены.

На этой диаграмме вся сеть Internet представлена узлом со стереотипом.

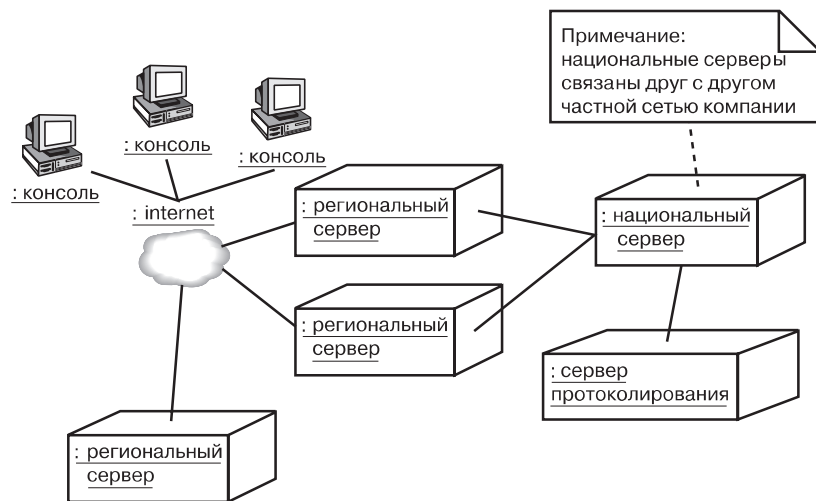


Рис. 31.4. Моделирование полностью распределенной системы

Прямое и обратное проектирование

Прямому проектированию (созданию кода по модели) диаграммы размещения поддаются лишь в очень небольшой степени. Например, после специфицирования физического распределения артефактов по узлам на диаграмме размещения можно воспользоваться инструментальными средствами, чтобы показать, как эти артефакты будут выглядеть в реальном мире. Для системных администраторов такое использование UML может оказать существенную помощь при решении весьма сложных задач.

Обратное проектирование (создание модели по коду) из реального мира в диаграммы размещения может иметь огромную ценность, особенно для полностью распределенных систем, которые постоянно изменяются. Чтобы приспособить UML к нужной предметной области, вы, вероятно, захотите подготовить набор узлов со стереотипами, понятных сетевому администратору. Преимущество использования UML состоит в том, что это стандартный язык, на котором можно выразить интересы не только администратора, но и разработчиков программного обеспечения.

Обратное проектирование диаграммы размещения следует осуществлять в таком порядке:

- ❑ Выбрать, что именно вы хотите подвергнуть обратному проектированию. В некоторых случаях вам нужно будет пройти по всей сети, но иногда достаточно ограничиться ее фрагментом.

- ❑ Выбрать степень детализации обратного проектирования. Зачастую работать можно на уровне всех процессоров системы, но бывает необходимо включить и сетевые периферийные устройства.
- ❑ Воспользоваться инструментальным средством, позволяющим совершить обход системы и показать ее топологию. Зафиксировать эту топологию в своей модели размещения.
- ❑ По ходу дела с помощью других инструментов выяснить, какие артефакты размещены в каждом узле, и занести их в модель размещения. Вам предстоит сложная поисковая работа, поскольку даже простейший персональный компьютер может содержать гигабайты разных артефактов, многие из которых не имеют отношения к вашей системе.
- ❑ Используя инструменты моделирования, создать диаграмму размещения путем опроса модели. Например, можно начать с визуализации базовой клиент-серверной топологии, а затем расширять диаграмму, добавляя в описания тех или иных узлов размещенные в них артефакты. Раскрыть детали содержания диаграммы размещения в той степени, в какой это требуется, чтобы донести ваши идеи до читателя.

Советы и подсказки

При создании в UML диаграмм размещения помните, что они являются всего лишь графическим представлением статического представления системы с точки зрения размещения. Это значит, что ни одна диаграмма, взятая сама по себе, не может описать все, что относится к размещению системы. Собранные вместе, диаграммы размещения дают полное представление о системе с соответствующей точки зрения; по отдельности же каждая диаграмма описывает лишь какой-то один аспект.

Хорошо структурированная диаграмма размещения обладает следующими свойствами:

- ❑ сосредоточена на каком-то одном аспекте статического представления системы с точки зрения размещения;
- ❑ содержит только те элементы, которые существенны для понимания данного аспекта;
- ❑ раскрывает только те детали, которые присутствуют на выбранном уровне абстракции, — показывает только те дополнения, которые существенны для понимания;
- ❑ не является настолько краткой, чтобы скрыть от читателя важную семантику.

Изображая диаграмму размещения, пользуйтесь следующими правилами:

- ☐ дайте диаграмме имя, соответствующее ее назначению;
- ☐ располагайте элементы так, чтобы число пересечений было минимальным и семантически близкие сущности оказывались рядом;
- ☐ используйте примечания и цветовые выделения, чтобы привлечь внимание к важным особенностям диаграммы;
- ☐ с осторожностью подходите к использованию элементов со стереотипами. Выберите ряд общих для вашего проекта или организации пиктограмм и применяйте их всюду и единообразно.

Глава 32. Системы и модели

В этой главе

- Системы, подсистемы, модели и представления
- Моделирование архитектуры системы
- Моделирование системы систем
- Организация артефактов разработки

UML – это графический язык для визуализации, специфицирования, конструирования и документирования артефактов программной системы. Его используют для того, чтобы моделировать системы. Модель – это упрощение реальности, абстракция, создаваемая, чтобы лучше понять систему. Система, зачастую разложенная на ряд подсистем, – это множество элементов, организованных некоторым образом для выполнения определенной цели. Система описывается набором моделей, по возможности рассматривающих ее с различных точек зрения. Важными составными частями модели являются такие сущности, как классы, интерфейсы, компоненты и узлы. В UML допускается моделирование систем и подсистем как единого целого – тем самым органично решается проблема масштабирования.

Хорошо структурированные модели помогают визуализировать, специфицировать, конструировать и документировать систему под разными (но вместе с тем взаимосвязанными) углами зрения. Хорошо структурированные системы функционально, логически и физически согласованы, но при этом состоят из слабо связанных друг с другом подсистем.

Введение

Для постройки собачьей будки серьезной подготовки не требуется. Потребности собаки просты, так что для их удовлетворения – если, конечно, не попался особо привередливый пес – вы просто сколачиваете будку без предварительного планирования.